Project Report

Script-Controlled ACL – Restrict Record Access Based on Field Value

Team ID: _____ *Team Members: _____*

1. INTRODUCTION

1.1 Project Overview

This project builds a script-controlled Access Control List (ACL) on the ServiceNow platform. The ACL script reads a field value on the record (for example, Sensitivity = Confidential) and the logged-in user, and returns true or false to allow or deny access. In this way a record can be restricted based on the data inside it, and not only on the user's roles.

1.2 Purpose

The purpose is to protect sensitive records from unauthorised users, reduce the number of extra roles and workarounds, and give administrators and compliance officers one clear and testable rule for record access.

2. IDEATION PHASE

2.1 Problem Statement

PS	Problem statement
PS-1	A ServiceNow administrator is trying to restrict access to records based on the value of a field, but role-based ACLs give access to every user holding the role, because ACL conditions without a script cannot check the field value against the user. This makes the administrator worried about data leaks and non-compliance.
PS-2	A security / compliance officer is trying to make sure only authorised users can read or edit sensitive records, but cannot easily prove who can access which record, because access logic is spread across roles, Business Rules and manual checks. This makes the officer unsure and frustrated during audits.

2.2 Empathy Map Canvas

The persona is a ServiceNow administrator / security officer who must make sure sensitive records are visible only to the right people.

Section	Summary
Says	“Why can everyone with the itil role open this confidential record?”, “Access should depend on the value of a field.”
Thinks	Is a script-based ACL safe and easy to maintain? What if the script blocks valid users?
Does	Creates many roles and groups, adds Business Rules, checks access manually and tests with impersonation.
Feels	Worried about data leakage and audit findings; confident when one clear script controls access.
Pains	Sensitive records visible to unauthorised users; many roles to maintain; hard to prove compliance.
Gains	Access restricted automatically by field value; fewer roles; consistent security; easy auditing.

2.3 Brainstorming

The team selected PS-1 and listed 11 ideas, grouped under access conditions, script logic, testing and audit, and user experience. The top priorities were:

Idea	Description	Impact	Effort
Idea 2	Confidential records for assigned user / caller only	High	Low
Idea 4	ACL script using gs.hasRole() and field value checks	High	Low
Idea 5	Reusable Script Include for ACL logic	High	Medium
Idea 7	Test with impersonation of different users	Medium	Low

3. REQUIREMENT ANALYSIS

3.1 Customer Journey map

The journey of the administrator has five stages: Identify (notice that role-only access is not enough), Design (select the table, field and who may access), Build (create the ACL script and Script Include), Test (impersonate users and check list, form and reports) and Maintain (update the rule and review it during audits). The full map is in the Requirement Analysis folder.

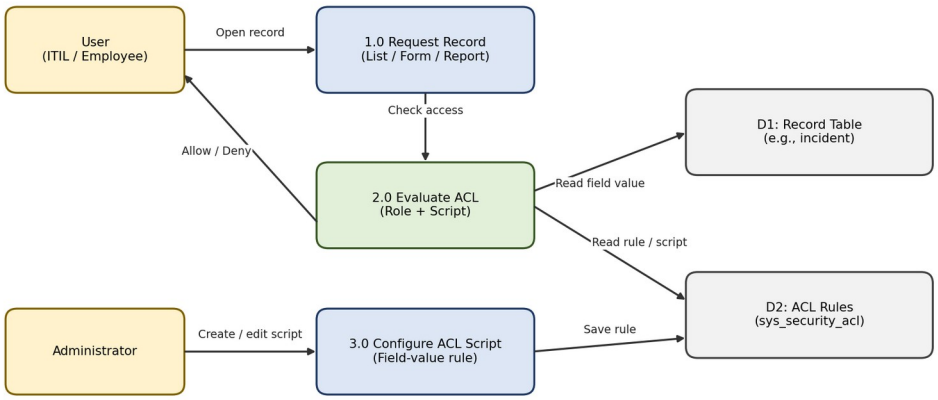
3.2 Solution Requirement

FR No.	Functional Requirement (Epic)	Sub Requirement
FR-1	Field-Value Based Record Restriction	Restrict access by field value; allow only authorised users
FR-2	Script-Controlled ACL	Record-level ACL with script returning true / false
FR-3	Operation Coverage	Read and write operations; list, form and reports
FR-4	Administration & Testing	Create in Studio / System Security; test with impersonation
FR-5	Reusable Access Logic	Keep the logic in a Script Include

Non-functional requirements: usability, security (deny by default), reliability (always return true / false), performance (lightweight script), availability (part of the platform) and scalability (reusable pattern).

3.3 Data Flow Diagram

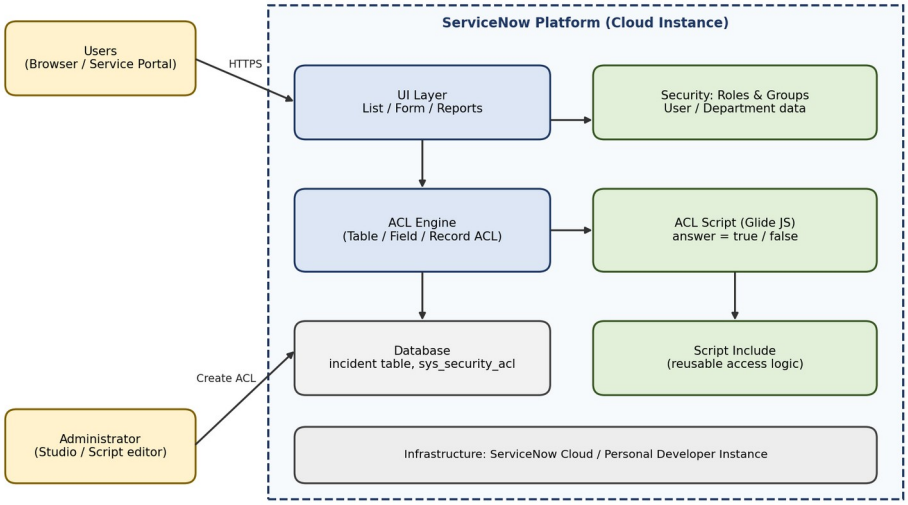
Data Flow Diagram (Level 1) – Script-Controlled ACL



3.4 Technology Stack

Component	Technology
User Interface	ServiceNow UI (Next Experience), Service Portal
Application logic	ACL script (Glide JavaScript), Script Include, gs.hasRole()
Database	ServiceNow tables: incident (example), sys_security_acl
Infrastructure	ServiceNow Personal Developer Instance (cloud)
Security	ServiceNow ACL, roles and groups

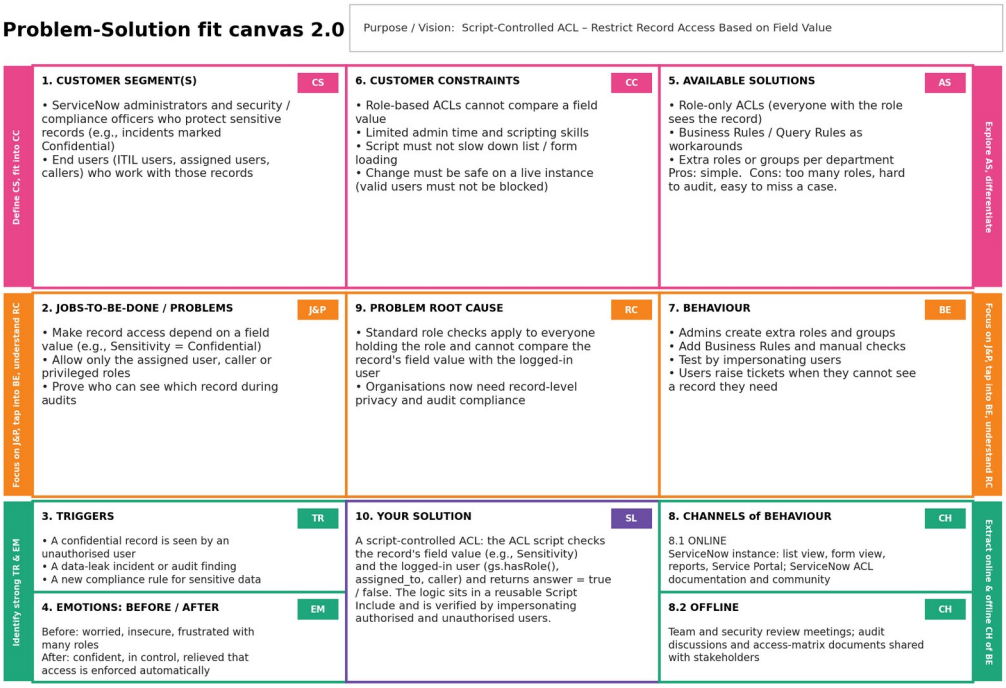
Technical Architecture – Script-Controlled ACL on ServiceNow



4. PROJECT DESIGN

4.1 Problem Solution Fit

The Problem-Solution Fit canvas matches the customer segment (administrators and security officers), their constraints, triggers and behaviour with the solution: a script-controlled ACL.



Problem-Solution fit canvas by Daria Nepriakhina / Amaltama.com – licensed under CC BY-NC-ND 4.0
Filled for: Script-Controlled ACL – Restrict Record Access Based on Field Value

4.2 Proposed Solution

Parameter	Description
Problem Statement	Role-based ACLs cannot check a field value, so sensitive records can be seen by users who should not see them.
Idea / Solution	Record-level ACL with a script that checks the field value and the user and returns true / false; logic kept in a Script Include.
Novelty	Access is decided by the data in the record; one scripted rule replaces many roles and workarounds.
Social Impact	Protects sensitive information and reduces data-leak risk.
Business Model	Built on the existing ServiceNow platform; value is lower security risk and less admin effort.
Scalability	Can be applied to other tables, fields and operations.

4.3 Solution Architecture

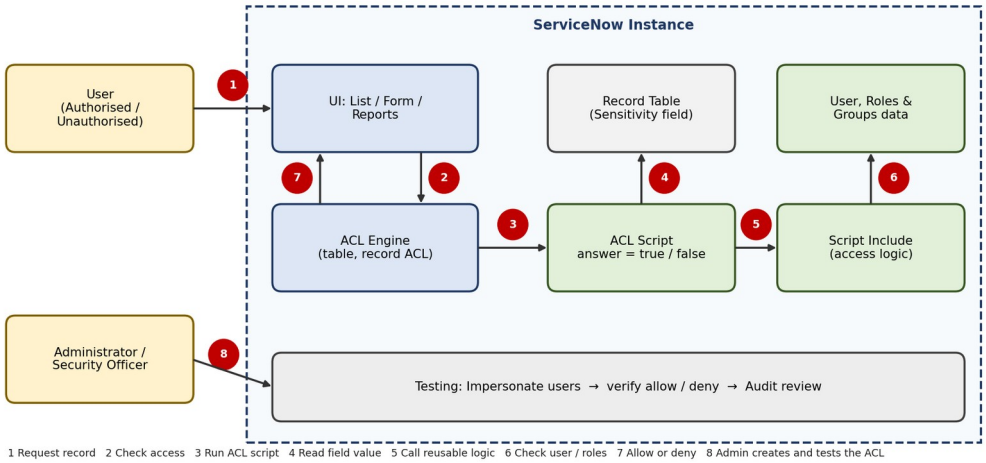


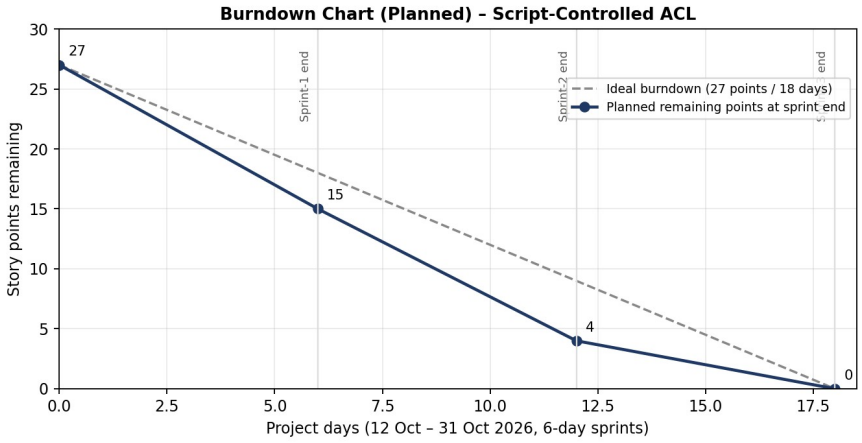
Figure: Architecture and data flow of the script-controlled ACL.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The work is planned in 3 sprints of 6 days, with 11 user stories and 27 story points in total. The planned velocity is 27 / 3 = 9 story points per sprint (1.5 points per day).

Sprint	Total Story Points	Duration	Sprint Start	Sprint End (Planned)
Sprint-1	12	6 Days	12 Oct 2026	17 Oct 2026
Sprint-2	11	6 Days	19 Oct 2026	24 Oct 2026
Sprint-3	4	6 Days	26 Oct 2026	31 Oct 2026



6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Testing is done by impersonating users. Nine functional tests (FT-01 to FT-09) check that authorised users are allowed and unauthorised users are denied in the form, list view, reports and write operation. Four performance tests (PT-01 to PT-04) check that records and lists open in under 3 seconds with the ACL active and that the script has no heavy queries. Twelve UAT test cases (TC-001 to TC-012) are also planned.

Test results: pending. The tests have not been executed yet; fill in the Actual Result and Pass/Fail columns of the test documents after testing.

7. RESULTS

7.1 Output Screenshots

[Add screenshots here after the ACL is built and tested: (1) the ACL record and script, (2) a Confidential record opened by an authorised user, (3) the same record denied for an unauthorised user, (4) the list view as an unauthorised user, (5) the impersonation test results.]

8. ADVANTAGES & DISADVANTAGES

Advantages

- Access depends on the field value in the record, not only on roles
- Fewer extra roles and workarounds to maintain
- Same restriction in list view, form view and reports
- One clear script that is easy to test and audit
- Reusable for other tables and fields

Disadvantages

- A wrong script can block valid users, so careful testing is needed
- Needs scripting skills and the security_admin role to change ACLs
- A heavy script can slow down list and form loading

9. CONCLUSION

A script-controlled ACL lets an organisation restrict record access based on a field value together with the identity of the user, which role-based ACLs alone cannot do. The planning, design and testing documents show how the rule can be built, tested by impersonation and reviewed during audits.

10. FUTURE SCOPE

- Apply the same pattern to more tables and field values
- Use security attributes and data conditions along with the script
- Log denied access attempts for audit reports
- Show a clear message to the user when a record is restricted

11. APPENDIX

Source Code (if any)

Example ACL script (illustrative; test it in your own instance):

```
// Record ACL (advanced) – table: incident, operation: read
// Example names (u_sensitivity, u_confidential_viewer): change them to match your
instance
answer = false;
var sensitivity = current.getValue('u_sensitivity');
if (sensitivity == 'public') {
    answer = true;
} else if (gs.hasRole('u_confidential_viewer') ||
           current.getValue('assigned_to') == gs.getUserID() ||
           current.getValue('caller_id') == gs.getUserID()) {
    answer = true;
}
```

Dataset Link: Not applicable (test records are created in the ServiceNow instance).

GitHub & Project Demo Link: [Add your link]

References:

https://www.servicenow.com/docs/r/yokohama/platform-security/access-control/exploring-access-control-list.html?contentId=oczJQGZK~1lGzH~ITrwZHg

https://www.servicenow.com/docs/r/oJu9n0q6rU~F9UHAnNCt5w/aGZqCO71IpYGqS0MTeJC7Q

https://www.servicenow.com/docs/r/application-development/servicenow-sdk/acl-api-now-ts.html